

LLM From Scratch — Learning Roadmap

A developer's guide to building, training and tweaking large language models

Phase 1 — Foundations (2–4 weeks)

Math prereqs

- Linear algebra: matrix ops, eigenvalues
 - Calculus: gradients, chain rule, Jacobians
 - Probability: cross-entropy, KL divergence
 - Information theory: entropy, likelihood
- [3Blue1Brown Linear Algebra + Calculus \(YouTube, free\)](#)

PyTorch fluency

- Tensors, autograd, custom nn.Module
 - Training loops, optimizers, schedulers
 - Mixed precision (bfloat16 / float16)
 - GPU memory model, .to(device)
- [PyTorch official tutorials \(free\)](#)

Phase 2 — Neural language models & transformers (3–5 weeks)

From bigrams to attention

- Bigram model → MLP → RNN → LSTM
 - Attention mechanism from first principles
 - Self-attention: Q, K, V matrices
 - Multi-head attention, positional encodings
- [Karpathy 'Zero to Hero' series \(YouTube, free\)](#)

Transformer architecture

- Encoder-decoder vs decoder-only (GPT)
 - Layer norm, residual streams, FFN
 - Causal masking, KV cache basics
 - Read: 'Attention Is All You Need' paper
- [Jay Alammar 'Illustrated Transformer' \(blog, free\)](#)

Phase 3 — Build a GPT from scratch (4–6 weeks)

Tokenization

- Character-level → BPE (Byte Pair Encoding)
 - SentencePiece, tiktoken internals
 - Vocab size tradeoffs
 - Implement BPE tokenizer yourself
 - Decode back to strings cleanly
- [Karpathy 'Let's Build the GPT Tokenizer' \(YouTube\)](#)

Build nanoGPT / microgpt

- Implement GPT block: attn + FFN + norm
 - Training loop: forward, loss, backward
 - Karpathy's microgpt: 200 lines of Python
 - Train on TinyShakespeare (CPU / free Colab)
 - Load GPT-2 weights, verify outputs match
- [github.com/karpathy/nanoGPT \(free\)](#)

Phase 4 — Pretraining at scale (4–8 weeks)

Data pipeline

- Web scrape / use Common Crawl, FineWeb
 - Deduplication: MinHash / LSH
 - Quality filtering, tokenize to .bin shards
 - DataLoader with shuffled shards
 - Chinchilla scaling law: $D^* = 20 \times N^*$
 - Pick model size based on FLOPs budget
- [Stanford CS336 lectures \(YouTube, free\)](#)

Efficient training

- AdamW, gradient clipping, LR warmup
 - Gradient accumulation for large batches
 - torch.compile, FlashAttention, bfloat16
 - Gradient checkpointing (memory vs speed)
 - DDP + FSDP for multi-GPU
 - Loss curves: detect overfit / underfit
- [nanochat repo by Karpathy \(free, ~\\$15-100 GPU\)](#)

Phase 5 — Post-training: SFT, RLHF, DPO (3–5 weeks)

Supervised fine-tuning (SFT)

- Format data as (prompt, response) pairs
 - Instruction-following datasets (SmolTalk)
 - Full fine-tune vs LoRA / QLoRA
 - Evaluate: ARC, MMLU, GSM8K benchmarks
 - Chat template formatting (system/user/asst)
- [mlabonne/llm-course GitHub \(free\)](#)

Alignment: RLHF & DPO

- RLHF: reward model + PPO loop
 - DPO: simpler, no reward model needed
 - GRPO (group relative policy opt.) for math
 - Constitutional AI / self-critique loop
 - Preference datasets: HH-RLHF, OpenHermes
- [nanochat GRPO pipeline \(free\)](#)

Phase 6 — Inference, evaluation & modern tweaks (ongoing)

Inference optimization

- KV cache: prefill vs decode phases
- Quantization: 8-bit, 4-bit (GPTQ, AWQ)
- Speculative decoding, continuous batching
- Sampling strategies: top-k, top-p, temp
- Deploy locally via llama.cpp or vLLM

→ [Karpathy nanochat inference engine \(free\)](#)

Modern architecture tweaks

- RoPE positional embeddings
- Grouped Query Attention (GQA)
- Mixture of Experts (MoE) layers
- SwiGLU activations, RMSNorm
- Read LLaMA, Mistral, DeepSeek papers

→ [arxiv papers \(all free\)](#)

Key free resources: Karpathy Zero to Hero (YouTube) • github.com/karpathy/nanoGPT • github.com/karpathy/nanochat • Stanford CS336 (YouTube) • github.com/mlabonne/llm-course • github.com/rasbt/LLMs-from-scratch